Ders 9 Çalışma Özeti

Ders 9 Çalışma Özeti

Genel Konular

- Yığın (stack) veri yapısı: LIFO düzeni, `push`, `pop`, `peek`/`top`, `isEmpty`, `isFull`.
- Stack overflow ve stack underflow durumları.
- Yığının kullanım alanları: fonksiyon çağrı yığını, ifade ayrıştırma, parantez kontrolü, ters çevirme, Hanoi Kuleleri, N-vezir.
- Yığının dizi ve bağlantılı liste (linked list) ile gerçeklenmesi.
- Örnekler: string'i ters çevirme, palindrom kontrolü, parantez dengesi kontrolü.
- Infix ifadeyi postfix'e çevirme (parantezleme yöntemi ve yığın yöntemi) ve postfix ifadeyi hesaplama.
- Kuyruk (queue) veri yapısı: FIFO düzeni, `enqueue`, `dequeue`.
- Kuyruğun dizi, dairesel dizi (circular array) ve linked list ile gerçeklenmesi.
- Yığın ve kuyruk işlemlerinin karmaşıklığı.

Hocanın Özellikle Vurguladığı Kısımlar

- "Yığın" kelimesi Türkçede bazen heap için de kullanılıyor. Karışmasın diye derste "stack" adı tercih edildi.
- Yığında yalnız en üstteki elemana erişilir. Ortadaki bir elemana doğrudan erişilemez.
- Temel işlemler `push` ve `pop`'tur. `isEmpty` ve `isFull` bunların güvenli çalışması için yardımcı fonksiyonlardır.
- `peek` nadiren gerekir. Yığındaki asıl amaç ekleme-çıkarma düzenini korumaktır.
- Dinamik yığın teoride dolmaz. Ancak fiziksel bellek sınırlı olduğu için pratikte stack overflow yaşanabilir.
- Derleyici tek yönde ilerlediği için infix ifadeyi işlem önceliğini gözeterek doğrudan hesaplayamaz. Postfix ifade soldan sağa tek geçişte hesaplanır.
- Yığın ve kuyruk işlemlerinin hepsi $O(1)$'dir. Algoritmalara ek karmaşıklık yükü getirmezler.
- Diziyle basit kuyrukta her `dequeue` sonrası kaydırma $O(n)$ sürer. Dairesel dizi bu maliyeti $O(1)$'e indirir.
- Hoca derste bir hesap hatası yaptı ve öğrencilerin hatayı ders sırasında hemen söylemesini istedi.

Kısa Tekrar Notları

- **Stack:** LIFO (last in, first out) ya da FILO (first in, last out).
- **Stack overflow:** dolu yığına `push`. **Stack underflow:** boş yığından `pop`.
- **Dizi ile stack:**
  - `top = -1` boş yığın demektir.
  - `push`: `top++`, sonra `stack[top] = veri`.
  - `pop`: `veri = stack[top]`, sonra `top--`.
- **Linked list ile stack:**
  - `top` işaretçisi listenin başını gösterir.
  - `push` başa ekleme, `pop` baştan silmedir.
- **Postfix'e çevirme (yığın ile):**
  - Operand doğrudan çıktıya yazılır.
  - `(` yığına itilir.

  - `)` gelince `(` görülene kadar yığından çekilip çıktıya yazılır.
  - Operatör gelince, yığının tepesindeki operatörün önceliği gelenin önceliğinden büyük ya da eşit olduğu sürece tepedeki çekilip çıktıya yazılır. Sonra gelen operatör itilir.
  - Girdi bitince yığında kalanlar sırayla çıktıya yazılır.
- **Postfix hesaplama:** Sayı yığına itilir. Operatör gelince iki sayı çekilir, işlem yapılır, sonuç tekrar itilir.
- **Queue:** FIFO (first in, first out). Ekleme `rear`'dan, çıkarma `front`'tan yapılır.
- **Dairesel kuyruk:** İndisler sona gelince başa döner: `rear = (rear + 1) % MAX`.
- **Kullanım alanları:**
  - Yığın: fonksiyon çağrıları, derleyicide ifade ayrıştırma, parantez kontrolü, geri izleme (backtracking).
  - Kuyruk: round robin işlemci zamanlaması, G/Ç işleme, router'da paket iletimi.

Detaylı Açıklamalar

Yığın Kavramı

Masaya üst üste konan kitaplar gibi düşünülür. Yalnız en üstteki kitap alınabilir. İlk konan kitap en son alınır.

Kapasitesi 6 olan bir yığına 1, 2, 3, 4, 5, 6 itilsin. Yığın dolar. 7'yi itmeye çalışmak stack overflow'dur. Sonra art arda `pop` yapılırsa 6, 5, 4, 3, 2, 1 sırasıyla çıkar. Boş yığından bir `pop` daha yapmak stack underflow'dur.

Fonksiyon Çağrı Yığını (Function Call Stack)

Bir fonksiyon çağrıldığında dönüş adresi ve yerel değişkenler bir paket olarak yığına itilir. Özyinelemeli fonksiyonda her çağrı yeni bir paket ekler. Temel duruma ulaşılınca paketler ters sırayla çekilir ve her çağrı kaldığı yerden devam eder. Yığın boşalınca `main`'e dönülmüş olur.

Dizi ile Yığın

`MAX` boyutlu dizi ve `top` indisi kullanılır. Başlangıçta `top = -1` olur.

```
int isEmpty(void) {
    if (top == -1) return 1;
    return 0;
}

int isFull(void) {
    if (top == MAX - 1) return 1;
    return 0;
}

int peek(void) {
    return stack[top];
}

void push(int data) {
    if (!isFull()) {
        top = top + 1;
        stack[top] = data;
    } else {
        printf("Stack full\n");
    }
}

int pop(void) {
    int data;
```

```
    if (!isEmpty()) {
        data = stack[top];
        top = top - 1;
        return data;
    }
    printf("Stack empty\n");
    return -1;   /* hata kodu */
}
```

`pop` sonrası değer dizide fiziksel olarak kalır. `top` aşağı indiği için artık kullanılmaz ve bir sonraki `push` o değerin üzerine yazar.

Linked List ile Yığın

`top` işaretçisi son eklenen düğümü gösterir. Her düğüm kendinden önce eklenen düğümü gösterir.

Slayttaki `push`:

```
void push(int data) {
    if (top == NULL) {
        top = (struct node *)malloc(1 * sizeof(struct node));
        top->ptr = NULL;
        top->data = data;
    } else {
        temp = (struct node *)malloc(1 * sizeof(struct node));
        temp->ptr = top;
        temp->data = data;
        top = temp;
    }
    count++;
}
```

`pop` için:

1. `top1 = top->ptr` ile bir sonraki düğüm saklanır.
2. Dönüş değeri `top->data`'dan alınır.
3. `free(top)` yapılır.
4. `top = top1` olur.

Yığın boşsa `-1` hata kodu döndürülür. `top1` olmadan `free` yapılırsa alttaki düğüm kaybedilir.

Diğer işlemler:

- `create`: `top = NULL`.
- `isEmpty`: `top == NULL`.
- `destroy`: düğümler sırayla `free` edilir.

Linked list yığını dolmadığı için `isFull` gerekmez. İstenirse bir sayaç ve üst sınırla eklenebilir.

String'i Ters Çevirme

"merhaba" karakter karakter, `'\0'` görülene kadar yığına itilir. Sonra yığın boşalana kadar `pop` yapılır: a, b, a, h, r, e, m. Sona `'\0'` eklenir. Maliyet $O(n)$'dir, ama kod çok sadedir.

Palindrom Kontrolü

Uzunluğu n olan string için ilk $\lfloor n/2 \rfloor$ karakter yığına itilir. n tekse ortadaki karakter atlanır. Kalan her karakter için bir `pop` yapılır ve karşılaştırılır.

Kelime	İtilen	Karşılaştırma	Sonuç
kabak	k, a (b atlanır)	a = a, k = k	Palindrom

Kelime	İtilen	Karşılaştırma	Sonuç
tabak	t, a (b atlanır)	a = a, k ≠ t	Palindrom değil (ilk uyuşmazlıkta durulur)
küllük	k, ü, l	l = l, ü = ü, k = k	Palindrom

Sonda string bitmiş ve yığın boşalmış olmalıdır.

Parantez Dengesi Kontrolü

Girdi soldan sağa okunur:

- Açma parantezi (`(`, `[`, `{`) yığına itilir.
- Kapama parantezi gelince bir `pop` yapılır. Türler eşleşmezse hata verilir.
- Girdi bitince yığın boş değilse parantezler dengesizdir.

Örnekler:

- `[ ( ) ]` ve sonrasında `{ }`: hepsi eşleşir, sonda yığın boştur. Doğru.
- `[ ( ( ) ]`: `]` gelince yığından `(` çıkar, eşleşmez. Hata.
- `[ ( ( ) )`: eşleşmeler doğru ama sonda yığında `[` kalır. Dengesiz.

Infix ve Postfix

İnsan infix yazar: `3 + 2 * 5 + 1 = 14`. Soldan sağa okuyan basit bir değerlendirici önceliği bilmezse $(3 + 2) \cdot 5 + 1 = 26$ bulur. Postfix gösterimde (`3 2 5 * + 1 +`) işlem sırası ifadenin içindedir.

C operatör öncelik tablosunda en yüksek öncelik parantez, dizi indisi ve `.`, `->` operatörlerindedir. Sonra sırasıyla:

1. artırma/azaltma,
2. tekli operatörler,
3. tip dönüşümü,
4. `* / %`,
5. `+ -`,
6. kaydırma,
7. ilişkisel operatörler,
8. eşitlik,
9. bit düzeyi ve mantıksal operatörler,
10. koşul ve atama.

Parantezleme yöntemi (uzun yol): İfade öncelik sırasına göre tamamen parantezlenir. Her operatör kendi kapama parantezinin yerine taşınır. Sonra bütün parantezler silinir.

$A/B-C+D*E-A*C$ ifadesi bu yöntemle `A B / C - D E * + A C * -` olur. Operandların sırası infix ve postfix'te aynı kalır.

Yığın yöntemi, örnek 1: `A * (B + C) / D`

Okunan	Yığın (alt → üst)	Çıktı
A		A
*	*	A
(	* (	A
B	* (	A B
+	* (+	A B
C	* (+	A B C
)	*	A B C +
/	/	A B C + *
D	/	A B C + * D
son		A B C + * D /

Örnek 2: `3 * (5 + 4 / 1) - 2` ifadesinin postfix'i `3 5 4 1 / + * 2 -` olur. Değeri $3 \cdot 9 - 2 = 25$'tir.

Postfix hesaplama: `6 2 / 3 - 4 2 * +`

Okunan	Yığın (alt → üst)
6, 2	6 2
/	3
3	3 3
-	0
4, 2	0 4 2
*	0 8
+	8

Sonuç 8'dir. Yani ifade $6/2 - 3 + 4 \cdot 2$'dir. Yığında sayılar yerine alt ifade string'leri tutulursa postfix tekrar infix'e çevrilmiş olur.

Örnek 3: $4 \wedge 2 * 3 - 3 + 8/4/(1+1)$

- Postfix: `4 2 ^ 3 * 3 - 8 4 / 1 1 + / +`
- Hesap: 16, 48, 45, $8/4 = 2$, $1 + 1 = 2$, $2/2 = 1$, $45 + 1 = 46$.

Hoca elle önce 49 bulmuştu. Hatası $8/4/2$ ifadesini $8/(4/2)$ gibi hesaplamaktı. Aynı öncelikli `/` operatörleri soldan sağa uygulanır: $(8/4)/2 = 1$. Doğru sonuç 46'dır.

Kuyruk (Queue)

Otobüs ya da uçak kuyruğu gibi ilk giren ilk çıkar. İşlemler `enqueue` (ekleme) ve `dequeue` (çıkarma) olarak adlandırılır.

Basit dizi ile kuyruk: Kapasite 10 olsun. 3, 5, 8, 1 sırayla eklenir. `dequeue` 3'ü çıkarır, sonra kalan elemanlar öne kaydırılır. Ekleme $O(1)$ ama çıkarma $O(n)$ olur.

Dairesel dizi ile kuyruk: `front` (çıkarılacak elemanın indisi), `rear` (son eklenen elemanın indisi) ve `size` tutulur. Kaydırma yapılmaz, yalnız indisler ilerler. `rear` dizinin sonuna gelince başa döner.

Örnek: `front = 6`, `rear = 9`, `size = 4`, kapasite 10. Yeni eleman 0. indise yazılır ve `rear = 0` olur.

Slayttaki `enqueue`:

```
void enqueue(int value) {
    if (size < MAXSIZE) {
        if (size < 0) {
            queue[0] = value;
            front = rear = 0;
            size = 1;
        } else if (rear == MAXSIZE - 1) {
            queue[0] = value;
            rear = 0;
            size++;
        } else {
            queue[rear + 1] = value;
            rear++;
            size++;
        }
    } else {
        printf("Queue is full\n");
    }
}
```

`dequeue` şöyle çalışır: kuyruk boşsa hata verir. Değilse `queue[front]` alınır, `size` azaltılır, `front` bir ilerletilir.

Linked list ile kuyruk: `count`, `front` ve `rear` bir `Queue` yapısında tutulur. Her düğüm `data` ve `next` içerir.

```
void enqueue(Queue *q, int value) {
    if (q->count < FULL) {
        Node *tmp;
        tmp = malloc(sizeof(Node));
        tmp->data = value;
        tmp->next = NULL;
        if (!isempty(q)) {
            q->rear->next = tmp;
            q->rear = tmp;
        } else {
            q->front = q->rear = tmp;
        }
        q->count++;
    } else {
        printf("List is full\n");
    }
}
```

dequeue adımları:

1. `tmp = q->front` ile çıkarılacak düğüm işaretlenir.
2. `n = q->front->data` ile değer saklanır.
3. `q->front = q->front->next` yapılır.
4. `count` azaltılır.
5. `free(tmp)` yapılır.

`init` ile `count = 0`, `front = rear = NULL` yapılır. `isEmpty` için `rear == NULL` kontrol edilir.

Karmaşıklık

İşlem	Stack (dizi / liste)	Queue (dairesel dizi / liste)
Ekleme (`push` / `enqueue`)	$O(1)$	$O(1)$
Çıkarma (`pop` / `dequeue`)	$O(1)$	$O(1)$
`peek`	$O(1)$	$O(1)$
`isEmpty` / `isFull`	$O(1)$	$O(1)$

Ders Sonrası Notlar

- **Dizi yığınında `isFull`:** Derste kontrol bir yerde `top == maxSize` diye anlatıldı. İndisler `0..MAX-1` olduğu için doğrusu `top == MAX - 1`'dir. Hoca da ilk anlatımda $n - 1$ demişti.
- **`^` operatörü:** Örnekte `^` üs alma olarak kullanıldı. C'de `^` üs alma değil, bit düzeyinde XOR'dur. Üs için `pow` kullanılır. Ayrıca üs alma genelde sağdan birleşir. Postfix'e çevirmede eşit öncelikli `^` için "büyük ya da eşitse çek" kuralı uygulanmaz.
- **Dairesel kuyrukta `size` başlangıcı:** Derste `size` başlangıçta -1, boşluk kontrolü `size < 0` olarak verildi. Bütün elemanlar çıkarıldıktan sonra `size` 0 olur. Bu durumda `isEmpty` yanlış sonuç verir ve `enqueue`'nun ilk dalı çalışmaz. Tutarlı çözüm `size`'ı 0 ile başlatıp boşluğu `size == 0` ile kontrol etmektir.
- **Dairesel kuyrukta `front`:** `dequeue` içinde `front` da başa dönmelidir: `front = (front + 1) % MAXSIZE`. Aynı şekilde `enqueue` tek satırda `rear = (rear + 1) % MAXSIZE` ile yazılabilir.
- **Linked list kuyrukta son elemanı çıkarma:** Son düğüm çıkarılınca `front` `NULL` olur ama `rear` eski düğümü göstermeye devam eder. `isEmpty` `rear`'a baktığı için hatalı sonuç verir. Çıkarmadan sonra `front == NULL` ise `rear = NULL` de yapılmalıdır.
- **Dizi ile basit kuyruk:** Kaydırmasız alternatif, yalnız `front` indisini ilerletmektir. Ancak o zaman dizinin başındaki boş yerler kullanılamaz. Dairesel dizi bu sorunu çözer.

Duyurular: - İlk ödev en geç ertesi gün sistemden tanımlanacak. Süre yaklaşık bir hafta olacak.
- Ders duyuruları takip edilmeli.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.